Centro Paula Souza

Fatec Jahu

ÁRVORES DE DECISÃO

APRENDIZAGEM DE MÁQUINA

CST em Desenvolvimento de Software Multiplataforma

PROF. Me. TIAGO A. SILVA

O QUE É O ALGORITMO ÁRVORE DE DECISÃO

- Imagine que você quer decidir se vai ou não sair para correr hoje. Seu processo mental pode ser algo assim:
  1) Primeira pergunta: "Está chovendo?"
     - Se Sim: A decisão é Não correr. Fim.
     - Se Não: Você continua para a próxima pergunta.
  2) Segunda pergunta: "A temperatura está acima de 30°C?"
     - Se Sim: A decisão é Não correr (muito quente). Fim.
     - Se Não: Você chega à decisão final: Sim, vou correr!
- Isso que você acabou de fazer é uma árvore de decisão.
- É um fluxograma onde cada "caixinha" interna é uma pergunta (um teste em um atributo) e cada caminho (ramo) representa a resposta a essa pergunta, levando a novas perguntas ou a uma decisão final.

O QUE É O ALGORITMO ÁRVORE DE DECISÃO

- Agora, aplicando essa ideia ao aprendizado de máquina: **a árvore de decisão é um modelo que aprende a tomar decisões criando um fluxograma a partir dos dados de treinamento**.

IMPLEMENTAÇÃO DO CÓDIGO

Implementando árvores de decisão com Python

CÉLULA 1: IMPORTANDO AS BIBLIOTECAS

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  import csv
```

CARREGAMENTO DOS DADOS

Carregando o dataset

CÉLULA 2: CARREGANDO OS DADOS DO DATASET

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```python
# Função para carregar dados do CSV
def carregar_dados(arquivo):
    X, y = [], []
    with open(arquivo, 'r') as f:
        leitor = csv.reader(f)
        next(leitor)  # pular cabeçalho
        for linha in leitor:
            X.append([float(linha[0]), float(linha[1])])  # idade, salario
            y.append(int(linha[2]))
    return np.array(X), np.array(y)

# Carregando o dataset
X, y = carregar_dados("dataset.csv")
print("Formato dos dados:", X.shape, y.shape)
print("Primeiras amostras:\n", X[:5], "\nRótulos:", y[:5])
```

CÉLULA 3: DEFINIÇÃO DA FUNÇÃO DE ENTROPIA

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```python
def entropia(y):
    classes, contagem = np.unique(y, return_counts=True)
    probs = contagem / len(y)
    return -np.sum(probs * np.log2(probs + 1e-9))

# Teste da entropia
print("Entropia [0,0,0,0] ->", entropia([0,0,0,0]))
print("Entropia [0,1,0,1] ->", entropia([0,1,0,1]))
```

O QUE A FUNÇÃO DE ENTROPIA FAZ?

- Na árvore de decisão, precisamos de um critério para medir o quão "puro" ou "misturado" está um conjunto de dados em relação às classes.
- Dois critérios muito usados são:
  - Índice Gini (que você já viu e até implementou).
  - Entropia (baseada na teoria da informação de Shannon).
- Interpretação:
  - Entropia = 0 onde o conjunto totalmente puro (todos os exemplos da mesma classe).
  - Entropia alta = onde o conjunto mais misturado (ex.: 50% de uma classe e 50% de outra → entropia máxima de 1).
- A função entropia mede a desordem/impureza de um conjunto de dados e guia a árvore de decisão para escolher divisões que deixem os nós mais “puros” possíveis.

CÉLULA 4: DIVISÃO DOS DADOS

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```python
def dividir(X, y, feature, limiar):
    esquerda_idx = X[:, feature] <= limiar
    direita_idx = X[:, feature] > limiar
    return X[esquerda_idx], y[esquerda_idx], X[direita_idx], y[direita_idx]

# Testando divisão por idade <= 40
Xe, ye, Xd, yd = dividir(X, y, feature=0, limiar=40)
print("Esquerda:", ye)
print("Direita:", yd)
```

CÉLULA 5: IMPLEMENTAÇÃO DA ÁRVORE DE DECISÃO

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```python
class No:
    def __init__(self, feature=None, limiar=None, esquerda=None, direita=None, valor=None):
        self.feature = feature
        self.limiar = limiar
        self.esquerda = esquerda
        self.direita = direita
        self.valor = valor  # classe se for folha
```

CÉLULA 5: IMPLEMENTAÇÃO DA ÁRVORE DE DECISÃO

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```python
class ArvoreDecisao:
    def __init__(self, max_profundidade=3, min_amostras=2):
        self.max_profundidade = max_profundidade
        self.min_amostras = min_amostras
        self.raiz = None

    def ajustar(self, X, y):
        self.raiz = self._crescer(X, y, profundidade=0)

    def _crescer(self, X, y, profundidade):
        num_amostras, num_features = X.shape
        num_classes = len(np.unique(y))
```

CÉLULA 5: IMPLEMENTAÇÃO DA ÁRVORE DE DECISÃO

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```
# Condições de parada
if (profundidade >= self.max_profundidade or
    num_classes == 1 or
    num_amostras < self.min_amostras):
    classe_final = np.bincount(y).argmax()
    return No(valor=classe_final)

melhor_feature, melhor_limiar, melhor_info = None, None, -1
melhor_divisao = None
```

CÉLULA 5: IMPLEMENTAÇÃO DA ÁRVORE DE DECISÃO

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```python
for feature in range(num_features):
    valores = np.unique(X[:, feature])
    for limiar in valores:
        X_esq, y_esq, X_dir, y_dir = dividir(X, y, feature, limiar)
        if len(y_esq) == 0 or len(y_dir) == 0:
            continue

        ganho = entropia(y) - (
            len(y_esq)/num_amostras * entropia(y_esq) +
            len(y_dir)/num_amostras * entropia(y_dir)
        )

        if ganho > melhor_info:
            melhor_feature, melhor_limiar, melhor_info = feature, limiar, ganho
            melhor_divisao = (X_esq, y_esq, X_dir, y_dir)
```

CÉLULA 5: IMPLEMENTAÇÃO DA ÁRVORE DE DECISÃO

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```
if melhor_info == -1:
    classe_final = np.bincount(y).argmax()
    return No(valor=classe_final)

esquerda = self._crescer(melhor_divisao[0], melhor_divisao[1], profundidade+1)
direita = self._crescer(melhor_divisao[2], melhor_divisao[3], profundidade+1)
return No(melhor_feature, melhor_limiar, esquerda, direita)
```

CÉLULA 5: IMPLEMENTAÇÃO DA ÁRVORE DE DECISÃO

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```python
def prever(self, X):
    return np.array([self._prever_amostra(x, self.raiz) for x in X])

def _prever_amostra(self, x, no):
    if no.valor is not None:
        return no.valor
    if x[no.feature] <= no.limiar:
        return self._prever_amostra(x, no.esquerda)
    else:
        return self._prever_amostra(x, no.direita)
```

CÉLULA 6: TREINANDO A ÁRVORE DE DECISÃO

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```
arvore = ArvoreDecisao(max_profundidade=3)
arvore.ajustar(X, y)

y_pred = arvore.prever(X)
acuracia = np.mean(y_pred == y)
print(f"Acurácia no conjunto de treino: {acuracia*100:.2f}%")
```

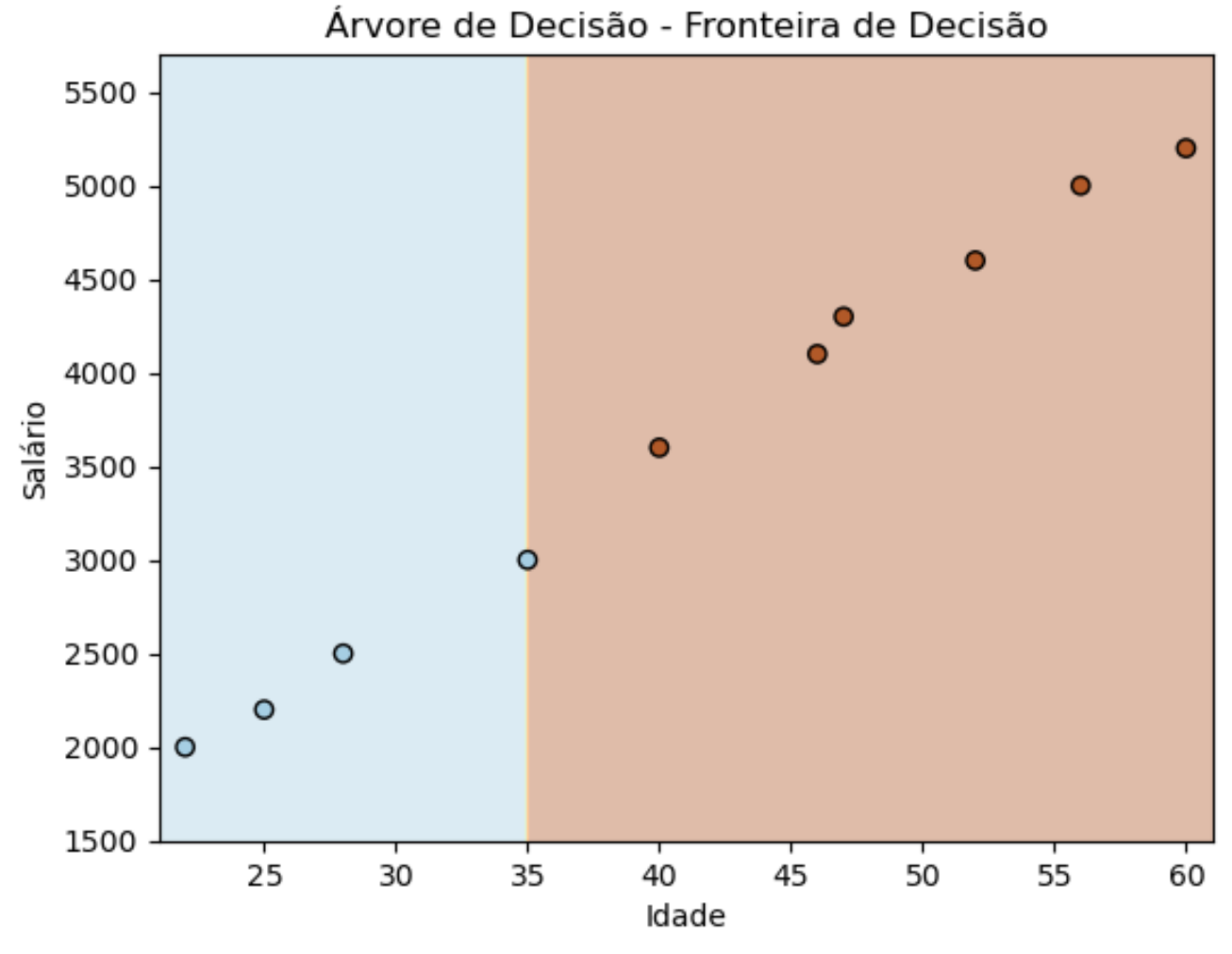
CÉLULA 7: PLOTANDO A FRONTEIRA DE DECISÃO

Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```
x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
y_min, y_max = X[:,1].min()-500, X[:,1].max()+500
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                     np.linspace(y_min, y_max, 200))
Z = arvore.prever(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.4, cmap=plt.cm.Paired)
plt.scatter(X[:,0], X[:,1], c=y, edgecolor="k", cmap=plt.cm.Paired)
plt.xlabel("Idade")
plt.ylabel("Salário")
plt.title("Árvore de Decisão - Fronteira de Decisão")
plt.show()
```

CÉLULA 7: PLOTANDO A FRONTEIRA DE DECISÃO



OBRIGADO!

- Encontre este **material on-line** em:
  - Slides: Google Classroom
- Em caso de **dúvidas**, entre em contato:
  - **Prof. Tiago**: tiago.silva@cps.sp.gov.br